

CORTEX: Stage 1 Parallel Development Workflow

This document describes **how the team works from Day 0 until the end of Stage 1**, ensuring that every member can begin development as early as possible while minimizing dependencies.

Team Members

Member	Responsibility
P1	Ingestion & Data Layer
P2	Retrieval & RAG Backend
P3	AI Agents & Reasoning
P4	Frontend, Auth & Infrastructure

Day 0: Planning & Contract Freeze

Duration: 2–4 Hours

No feature development begins before this phase is completed.

Step 1: Architecture Planning (Everyone)

The team decides:

- Overall project architecture
- Folder structure
- Git workflow
- Branch naming strategy
- Coding standards
- Tech stack

- Repository organization

Example:

```
backend/  
frontend/  
docker/  
docs/
```

Step 2: Freeze Database Schema (P1)

P1 designs and publishes the database schema.

Example tables:

- document_registry
- facts
- entity_registry
- users
- audit_log

Once finalized, every developer builds against this schema.

Step 3: Freeze Neo4j Ontology (P1)

P1 defines:

Node Types

- Equipment
- Failure
- Specification
- Document
- Manual

Relationship Types

- CONNECTED_TO
- HAS_SPEC
- HAS_FAILURE

- MENTIONS

Everyone now knows the exact graph structure.

Step 4: Freeze API Contracts (P2 + P3)

P2 and P3 jointly define all backend APIs.

Examples:

- POST `/api/query`
- POST `/api/agents/asset`
- POST `/api/agents/diagnose`
- POST `/api/agents/comply`

They finalize:

- Request JSON
- Response JSON
- Error responses
- Streaming (SSE) event names

These interfaces remain stable throughout development.

Step 5: Freeze Shared Function Contracts (P2 + P3)

Shared functions are defined before implementation.

Example:

```
context_graph_query(  
    tag,  
    query,  
    depth  
)
```

The input and output format is frozen.

Internal implementation can change later without affecting other modules.

Step 6: Create Development Fixture (P1)

P1 creates a small but realistic dataset containing:

- 10 fake equipment
- 30–50 knowledge facts
- Neo4j nodes and relationships
- Sample embeddings in Qdrant
- Sample citations

This fixture allows every developer to start immediately without waiting for the complete ingestion pipeline.

Step 7: Docker Environment (P4)

P4 prepares Docker Compose for the complete development environment.

Services include:

- PostgreSQL
- Neo4j
- Qdrant
- Redis
- Backend
- Frontend

Everyone now works in an identical environment.

End of Day 0

At this point:

- Database schema is frozen
- Graph ontology is frozen
- API contracts are frozen
- Function signatures are frozen
- Docker environment is running
- Development fixture is available

Now all four developers begin working independently.

Week 1: Independent Development

P1: Ingestion & Data Layer

Develops:

- Upload API
- PDF handling
- OCR
- Docling integration
- Excel parsing
- Regex extraction
- LLM structured extraction
- Entity resolution
- PostgreSQL writer
- Neo4j writer
- Qdrant writer

P1 works independently without depending on other team members.

P2: Retrieval & RAG Backend

Uses the development fixture.

Builds:

- Query classification
- Hybrid retrieval
- PostgreSQL search
- Neo4j traversal
- Qdrant semantic search
- Fusion
- Cross-encoder reranking
- Citation generation
- `/api/query`

Since the fixture already exists, P2 does not wait for real document ingestion.

P3: AI Agents & Reasoning

Builds:

- LangGraph supervisor
- Asset Agent
- Diagnose Agent
- Comply Agent

Initially,

`context_graph_query()` returns mocked graph data.

This allows:

- Supervisor testing
- Routing validation
- Agent execution
- Logging
- Streaming

without waiting for real retrieval.

P4: Frontend, Auth & Infrastructure

Builds:

- Login
- Dashboard
- Upload page
- Chat interface
- Graph Explorer
- Equipment pages
- Citation viewer
- Authentication
- Role-based access

Uses mocked API responses and mocked SSE events.

No backend dependency exists.

End of Week 1: First Demonstration

Each developer demonstrates their own vertical slice.

P1

- Upload one document
 - Parse successfully
 - Store extracted data
-

P2

- Execute queries
 - Retrieve information from the fixture
 - Generate citations
-

P3

- Supervisor routes requests correctly
 - Agents execute using mocked retrieval
 - Logging works
-

P4

- Complete frontend navigation
 - Streaming UI works
 - Graph visualization works
-

Week 2: Integration Phase

Now each developer replaces mocks with real implementations.

P1

Improves:

- Real extraction
 - More documents
 - Better entity resolution
 - Better embeddings
-

P2

Changes data source from:

Fixture Database

to

Real Database

No code rewrite is required because the schema remains unchanged.

P3

Changes:

Mocked `context_graph_query()`

to

Real `context_graph_query()`

The function signature remains identical.

No changes are required in the agents.

P4

Changes:

Mock APIs

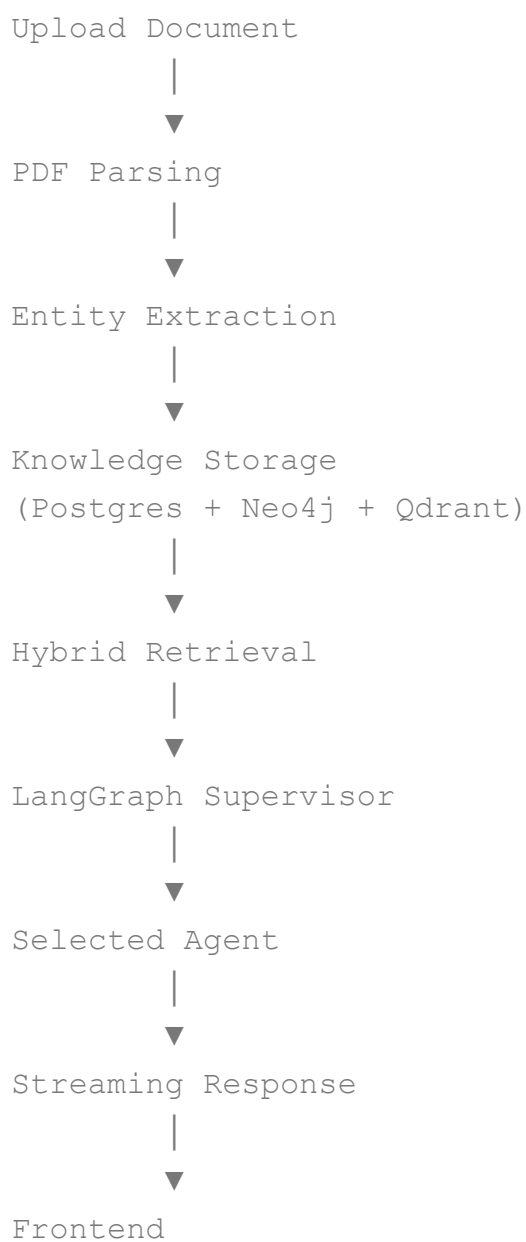
to

Real backend APIs

Since the JSON responses match the agreed contracts, only configuration changes are needed.

Integration Testing

The complete pipeline is tested.



Bugs discovered during integration are fixed collaboratively.

Week 3: Optimization & Polish

P1

- Improve extraction quality
 - Improve entity resolution
 - Optimize parsing speed
-

P2

- Tune hybrid retrieval
 - Improve ranking
 - Improve citation quality
 - Optimize latency
-

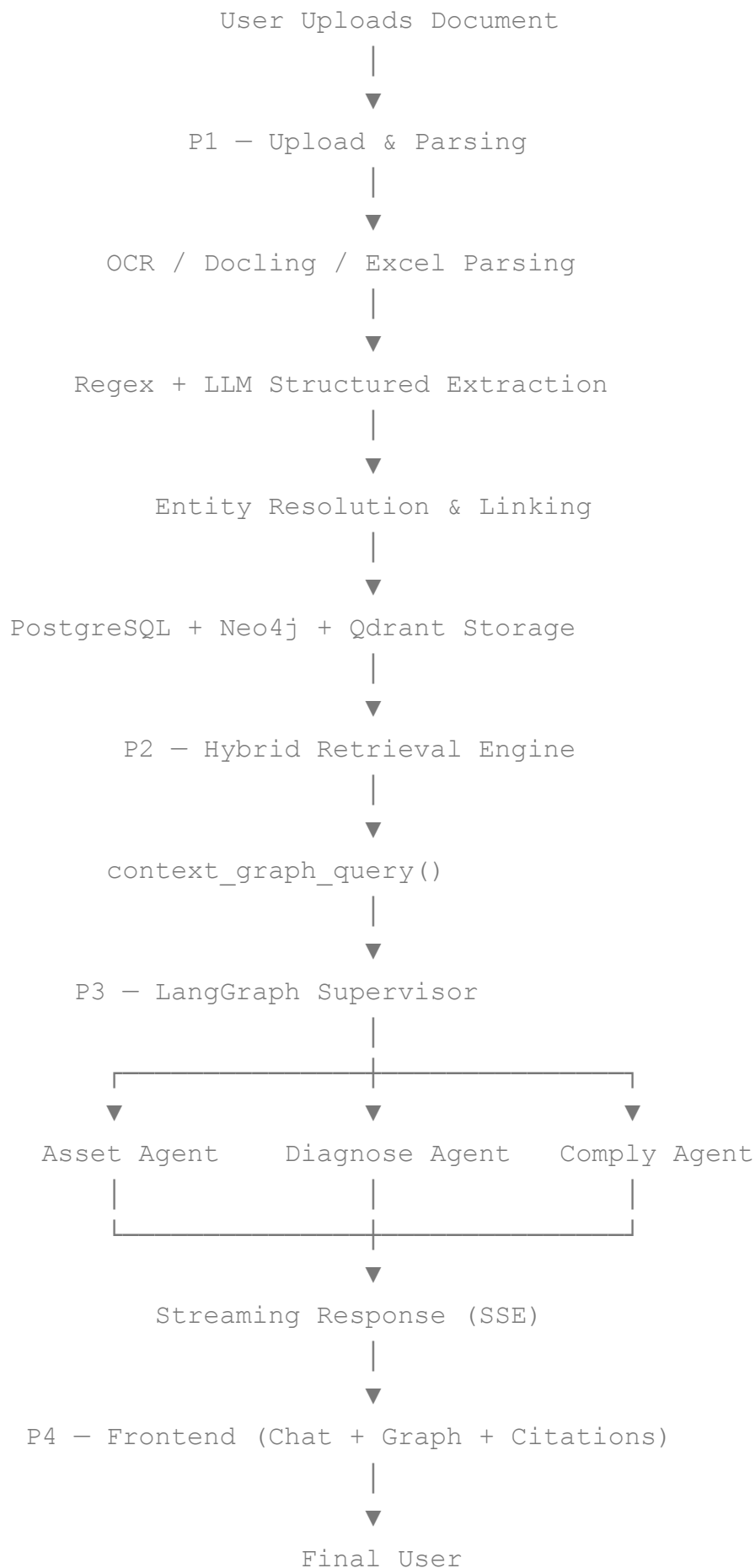
P3

- Improve prompts
 - Improve reasoning
 - Improve routing
 - Improve logging
-

P4

- Improve UI
 - Improve UX
 - Add loading states
 - Better error handling
 - Deployment
 - Final testing
-

Final Stage 1 Pipeline



Timeline Summary

Time	P1	P2	P3	P4
Day 0	Freeze database schema, ontology, fixture	Freeze API contracts	Freeze agent contracts & shared functions	Docker Compose, project scaffold
Week 1	Build ingestion pipeline	Build retrieval using fixture	Build agents using mocked retrieval	Build UI using mocked APIs
End of Week 1	Demo ingestion	Demo retrieval	Demo agents	Demo frontend
Week 2	Improve extraction	Switch to real data	Switch to real retrieval	Switch to real backend
Week 3	Optimize extraction	Optimize retrieval	Optimize reasoning	Polish UI & deploy

Key Principle

The team does **not** depend on unfinished code from other developers.

Instead, everyone depends only on **frozen contracts**:

- Database schema
- Graph ontology
- API specifications
- Shared function signatures
- Development fixtures

As long as these interfaces remain unchanged, each developer can build, test, and iterate independently, enabling true parallel development throughout Stage 1.